Modern Paradigms in Learning and Decision-Making
Lecture #1: Class Overview and Introduction to Indistinguishability

Juan C. Perdomo

Fall 2026

The impact of data-driven algorithms on society is increasingly vast. They shape the content we see online, the routes we take to work, the prices of the goods we buy, and even the people we meet and choose to date. The scale at which modern predictive systems operate and the degree to which they are intertwined with their encompassing social environments raise a universe of exciting possibilities and concerns.

We would love to be able to apply pattern recognition systems at a national scale to identify who is going to be sick in the future and be able to effectively intervene on them today. We want to automatically discover which kinds of educational interventions work best on different students, to automate away tedious bureaucratic decisions to make way for human discretion, and to provide personalized advice that enables people to learn about their own behavior and improve their decision-making. At the same time, there is the risk that algorithms replicate historical biases, reduce individual agency, and perpetuate harm. The ease with which we can now build and deploy predictive or GenAI tools places newfound pressure to guard against these concerns and develop formal guarantees that the systems we build are aligned with broader social values.

This class approaches these sets of questions from a formal, theoretical perspective. Mathematics is the language of precision. The class will provide an introduction to recent mathematical frameworks and definitions that each aim to precisely name and address a particular concern within this broader space of algorithms and society. The goal is to develop a formal language that allows us to very clearly talk about what reliability properties we can demand algorithmic systems to have in specific contexts, and which ones we cannot. We will also look into theory for the sake of theory.

1 Sets of Topics

The class will consist of three main modules each covering a particular set of topics.

- **Prediction and Learning.** Given a particular object that we are interested in studying, for instance the relationship between patient demographics and future disease or the mapping from the description of an image to its visual representation, how do we "learn" it? What does it really mean to make a prediction or forecast a probability for an event that only happens once? How do we make predictions in the presence of feedback loops where the prediction changes the likelihood of the outcome itself? How do we make predictions if the population is constantly changing, or ensure that they are valid for every group of people?

- **Decision-Making.** More often than not, the reason why we make predictions is because we want to effect some kind of outcome and make a decision. We predict travel times to decide which way to get to work, and forecast disease to decide on interventions today. Clearly, an oracle that tells us exactly what is going to happen in the future is sufficient to make a good decision. But, given that we can never hope to learn things perfectly, what level of accuracy is really necessary to ensure our decisions are "good" (where good is appropriately defined)?
- **Verification.** The last set of topics will be relatively more exploratory. Motivated by how easy it is nowadays to have agents build models or acommplish different tasks with little oversight, the goal of this section is to understand methods for reliably and efficiently checking that whatever learning task was delegated was indeed solved appropriately.

2 Warmup: Indistinguishability & Boosting

Learning is a form of approximation.

Given an unknown object p^* we want to compute a model p that is "close" to the p^*. Ideally, we would like p to be indistinguishable from p^*. That is, no one should be able to tell the difference between the true object, and our model of that object. Indistinguishability has deep roots within theoretical computer science and can take different forms depending on the setting. Today, we will focus on one of the simplest cases to build some intuition behind the techniques.

Let $p^* : \mathcal{X} \to \mathbb{R}$ be a function from a finite set of real numbers $\mathcal{X} = \{x_1, \ldots, x_n : x_i \in \mathbb{R}\}$ to the real line. Furthermore, let $\mathcal{F} = \{f_1, \ldots, f_k\}$ be a collection of real-valued functions $f_i : \mathcal{X} \to \mathbb{R}$. We say that a model p is indistinguishable from p^* with respect to $\mathcal{F}$ if no function $f \in \mathcal{F}$ can tell the difference between p and p^*. More formally,

Definition 2.1 (Indistinguishability)**.** *We say p is ε-indistinguishable from p^* with respect to $\mathcal{F}$ if*

$$\left| \frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} f(x)(p(x) - p^*(x)) \right| \leq \varepsilon \qquad \text{for every } f \in \mathcal{F}.$$

To build some intuition, a model p is indistinguishable with respect to a specific function f if on average over the domain $\mathcal{X}$, $f(x)$ is uncorrelated with the residual $p(x) - p^*(x)$. To be indistinguishable with respect to a *set* of functions $\mathcal{F}$ this condition must hold for every function $f \in \mathcal{F}$. Intuitively, the richer class of functions $\mathcal{F}$ is, the closer p needs to be to p^* in order to remain indistinguishable. For instance,

- Suppose $\mathcal{F}$ only contains that constant function, $\mathcal{F} = \{x \to 1\}$. Then, a very simple model that always outputs the average value of p^* over the domain $\mathcal{X}$ is 0-indistinguishable,

  $$p(x) = \frac{1}{|\mathcal{X}|} \sum_{x' \in \mathcal{X}} p^*(x')$$

  However, because the indicator function is unexpressive, there are lots of other models that are also indistinguishable. For instance, any function that has the same average value over the domain is also indistinguishable with respect to the constant function. See Figure 1.

- On the other hand, if $\mathcal{F}$ contains all possible indicator functions $\mathcal{F} = \{x \to 1\{x = x_i\} : x_i \in \mathcal{X}\}$, then the only function that is 0-indistinguishable from p^* is p^* itself. As we change how rich $\mathcal{F}$ is we can interpolate between both extremes.

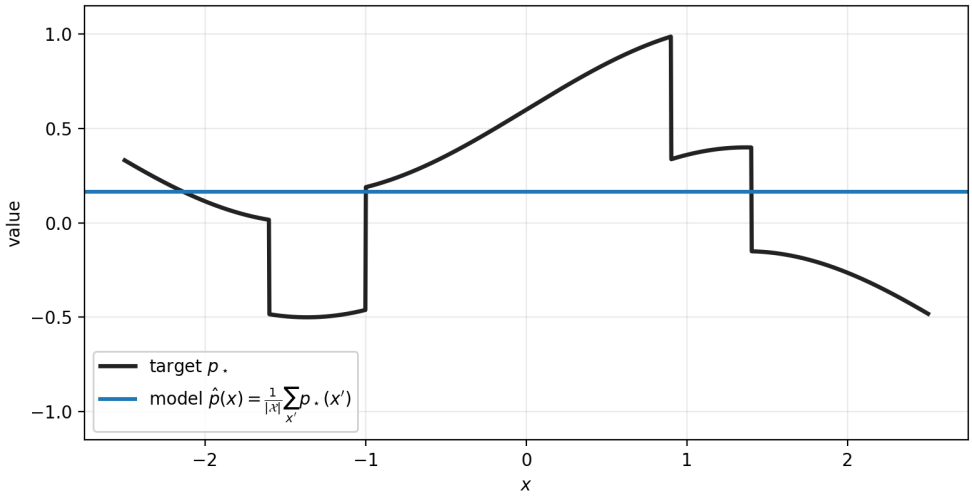

Figure 1: A target p^* and the constant model p that outputs the average value of p^* over $\mathcal{X}$. With respect to $\mathcal{F} = \{x \to 1\}$, this horizontal line is 0-indistinguishable from p^*: the single test in $\mathcal{F}$ cannot tell them apart, even though the two functions look nothing alike.

2.1 Finding an Indistinguishable Predictor

As we will see throughout the course, indistinguishability is a very powerful notion with a rich array of consequences for learning and decision-making. Before we get to those, we will first examine the question of finding an indistinguishable predictor.

Of course, one might ask, given that we have some kind of access to p^*, why should we be concerned with finding a model of p^*? There are numerous reasons, but the primary concern is that p^* is often a very complex object that is infeasible to compute and write down a circuit for.

For instance, p^* will often be equal to the conditional expectation of a distribution, $p^*(x) = \mathbf{E}[Y \mid X = x]$. Here, we cannot even access p^* for every x. At best we could draw samples from the underlying distribution. Furthermore, for very large domains $\mathcal{X}$, for instance $\mathcal{X} = \{\pm 1\}^n$ (which is exponentially large in n), it is infeasible to have code that matches p^* everywhere.

While p^* is complex, we will think of the class $\mathcal{F}$ as a large class of simple functions that we can efficiently search over. For example, we can think of $\mathcal{F}$ as

$$\mathcal{F} = \{x \to x, x \to x^2, x \to 1\{x > 0\}, x \to x \cos x\}$$

The key benefit of indistinguishability is that regardless of how rich or complex p^* is, for any class $\mathcal{F}$, there is always a model p that is indistinguishable from p^* from the view of $\mathcal{F}$ while being not

much more complicated than any function in $\mathcal{F}$. Importantly, the complexity of the model p is *independent* of p^*.

Furthermore, there is a very simple boosting procedure due to TTV [TTV09] that can find such a model p. It relies on one core search procedure:

```
AUDIT(p, F, eps)
    IF there is f in F such that p is not eps-indistinguishable with respect to f
        return f
    ELSE
        return PASS
```

This AUDIT procedure is nothing else than just a search algorithm that checks whether there is a function f that witnesses a difference between p and p^*. For now, we will ignore the question of whether there is an *efficient* implementation of this audit procedure that works for the kinds of $\mathcal{F}$ that we care about in practice, and just assume that we can call an oracle. Given such an oracle, the following routine yields an indistinguishable model,

```
BOOST(F, eps):
Initialize p_0(x) = 0 for all x
For t=0,1,2,...
    Call AUDIT(F, p_t, eps)
    If AUDIT returns a function f_t, update p_{t+1} = p_t - eta_t f_t
    If AUDIT returns PASS, return p_t
```

Notice that if the BOOST algorithm ever terminates, it must be the case that the model p_t is indistinguishable from p^* with respect to $\mathcal{F}$. The AUDIT subroutine only returns PASS if this condition is true. The main question is therefore not whether the boosting procedure would return a bad model, but whether it would ever terminate at all.

To show termination, we need to show that the indistinguishability condition is met after some bounded number of iterations. The key insight is that, when viewed from the right perspective, each iteration makes sufficient progress along a particular notion of distance between p_t and p^*. Since distances cannot go below 0, and each update makes constant progress, the algorithm must therefore halt. This is known as a potential argument.

Proposition 2.2. *Assume that* $\sup_x |p^*(x)| \leq 1$ *and that* $\sup_{f\in\mathcal{F},x\in\mathcal{X}} f(x)^2 \leq 1$ *then, the BOOST procedure halts after at most* $1/\varepsilon^2$ *many iterations and returns a model* p *that is* ε*-indistinguishable with respect to* $\mathcal{F}$*.*

Proof. Define the inner product

$$\langle f, g\rangle = \frac{1}{|\mathcal{X}|}\sum_{x\in\mathcal{X}} f(x)g(x)$$

with norm $\|f\|^2 = \langle f, f\rangle$ and define the potential function $\Phi_t = \|p_t - p^*\|^2$. Using this notation,

every time the boosting procedure makes an update it must be the case that

$$|\langle f_t, p_t - p^* \rangle| = \left| \frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} f_t(x)(p_t(x) - p^*(x)) \right| \geq \varepsilon$$

Furthermore, whenever we make an update and set $\eta_t = \varepsilon \cdot \text{sign}(\langle f_t, p_t - p^* \rangle)$

$$\begin{aligned}
\Phi_{t+1} &= \Phi_t - 2\eta_t \langle f_t, p_t - p^* \rangle + \eta_t^2 \|f_t\|^2 \\
&= \Phi_t - 2\varepsilon |\langle f_t, p_t - p^* \rangle| + \varepsilon^2 \|f_t\|^2 \\
&\leq \Phi_t - 2\varepsilon^2 + \varepsilon^2 \\
&\leq \Phi_t - \varepsilon^2
\end{aligned}$$

Therefore, each step reduced Φ by ε^2. Since $\Phi_0 = \|p^*\|^2 \leq 1$ and $\Phi_t \geq 0$ for every t, we get that the procedure must halt after $1/\varepsilon^2$ many updates. □

There are several interesting things worth pointing out from the algorithm. First, every model update is of the form, $p_{t+1} = p_t - \eta_t f_t$. Therefore, if we unroll these updates across time, we see that the resulting model p is a linear combination of at most $\lceil 1/\varepsilon^2 \rceil$ many functions in $\mathcal{F}$,

$$p(x) = \sum_t \eta_t f_t(x).$$

It doesn't matter if $\mathcal{F}$ has just 4 functions or 4 million functions. There always exists a linear combination of size at most $1/\varepsilon^2$, a quantity that is independent of $|\mathcal{F}|$, that is ε-indistinguishable from with respect to $\mathcal{F}$. This holds for any class of bounded functions $\mathcal{F}$ and underlying target p^*!

Furthermore, it was important, for the sake of the analysis, that we chose the potential function to be the squared distance. This enabled us to form a simple proof that illustrated how the updates make progress on every step. This monotonicity property is not true if we instead measured progress using the indistinguishability error,

$$\max_{f \in \mathcal{F}} \left| \frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} f(x)(p(x) - p^*(x)) \right|,$$

as illustrated by the following animation.

2.2 Stronger Notions of Indistinguishability

Notice that indistinguishability is a matter of perspective. Models that are indistinguishable with respect to one set of tests $\mathcal{F}$ may not be indistinguishable with respect to richer classes of tests $\mathcal{F}'$. Indistinguishability also becomes stronger and more interesting to the extent that we allow greater access to the underlying model.

In particular, the functions f we've considered so far can only falsify models by examining differences $p^*(x) - p(x)$ across a weighted subset of the domain. One way to strengthen the guarantee is to allow f to examine not just x, but also the model outputs $p(x)$.

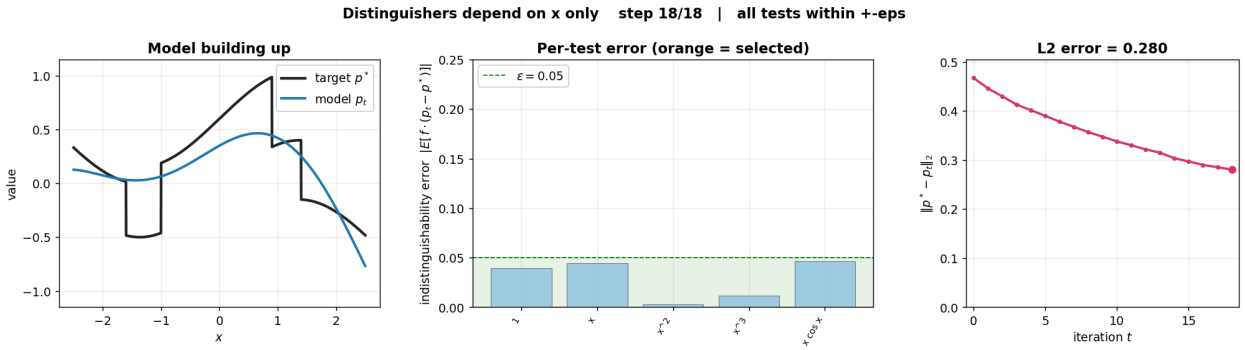

Figure 2: Final state of the BOOST procedure run with $\varepsilon = 0.05$ and the class of tests $\mathcal{F} = \{1, x, x^2, x^3, x\cos x\}$, which depend only on x. Left: the target and the returned model. Middle: the indistinguishability error of each test, all inside the $\pm\varepsilon$ band at termination. Right: the squared-distance potential decreases monotonically across iterations, even though the per-test errors do not.

Now let $\mathcal{F}$ be a set of functions $f : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$. We say that a model is ε-output indistinguishable if

$$\max_{f \in \mathcal{F}} \left| \frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} f(x, p(x))(p(x) - p^*(x)) \right| \leq \varepsilon.$$

Allowing the functions f to also take in $p(x)$ is, as we will see, significantly more powerful. It allows us to verify that the model is self-consistent and guarantee for instance that $p(x)$ is equal to $p^*(x)$ even conditional on the output values of p,

$$\left| \frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} 1\{p(x) = v\}(p(x) - p^*(x)) \right| \leq \varepsilon.$$

Furthermore, the exact same boosting argument we saw before also works for this setting. The proof goes through word for word.

Proposition 2.3. *Fix a class of functions $\mathcal{F} \subset \{\mathbb{R} \times \mathbb{R} \to \mathbb{R}\}$. Assume that $\sup_x |p^*(x)| \leq 1$ and that for every $f \in \mathcal{F}$, $\sup_{x \in \mathcal{X}} f(x)^2 \leq 1$. Then, the BOOST procedure halts after at most $1/\varepsilon^2$ many iterations and returns a model p that is ε-output indistinguishable with respect to $\mathcal{F}$.*

The crucial difference relative to the first setting is that the resulting model p can no longer be written as a linear combination of the functions f in $\mathcal{F}$. The updates in this case are of the form,

$$p_{t+1}(x) = p_t(x) - \eta_t f_t(x_t, p_t(x_t)).$$

Given that f now takes in the tth model as input, we can no longer unroll the recursion. In this case, we need to compute outputs via dynamic programming. Starting at level 0, we compute p_0 and the compute p_1 by using the function f_1 returned at the first iteration of the boosting procedure and so on. Visually, we now have a "deep" circuit of depth $1/\varepsilon^2$, rather than a wide circuit of depth 1 and width $1/\varepsilon^2$. It's worth visualizing the differences in this animation.

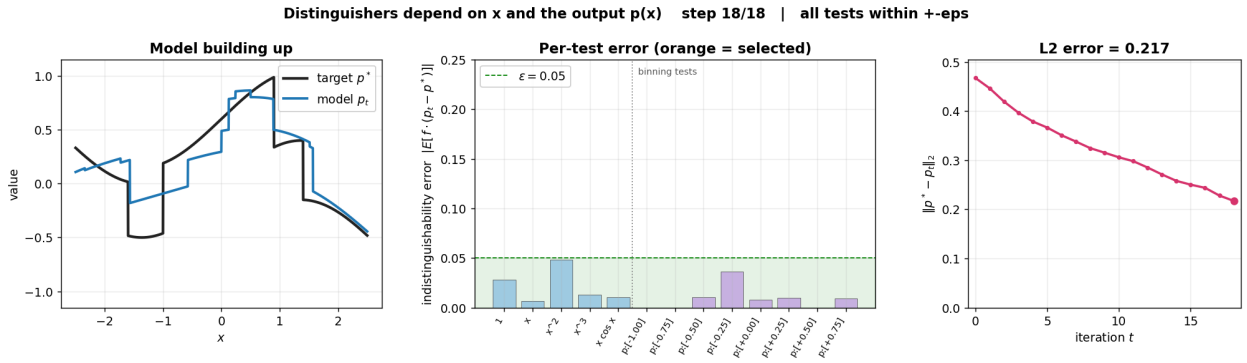

Figure 3: Final state of the BOOST procedure run with $\varepsilon = 0.05$ on the same target as Figure 2, but where the class of tests now also contains functions of the model's output: the level-set indicators $1\{p(x) \in B\}$ for buckets B partitioning $[-1, 1]$, alongside the five tests that depend only on x. At termination every test in the larger class is inside the $\pm\varepsilon$ band, and the model achieves a lower squared distance to the target (0.217 versus 0.280 in Figure 2).

References

[TTV09] Luca Trevisan, Madhur Tulsiani, and Salil Vadhan. Regularity, boosting, and efficiently simulating every high-entropy distribution. In *Proceedings of the 24th Annual IEEE Conference on Computational Complexity (CCC)*, pages 126–136, 2009.